Number Theory II

Modular arithmetic, fast power, inverses & nCr mod p

Learning goals

- Why answers are asked **"modulo $10^9 + 7$"**
- Add, subtract, multiply mod m **without overflow or negatives**
- **Binary exponentiation**: a^b mod m in O(log b)
- **Modular inverse** via Fermat's little theorem (and when it fails)
- Precompute **factorials and inverse factorials** → nCr in O(1)
- Classic counting: stars and bars, Catalan numbers, grid paths
- Reduce huge exponents with **mod p − 1**

Motivation: CSES 1079 — Binomial Coefficients

$n \le 10^5$ queries: print $C(a, b) \bmod 10^9 + 7$, with $0 \le b \le a \le 10^6$.

```
3
5 3   → 10
2 1   → 2
10 5  → 252
```

- $C(10^6, 5 \cdot 10^5)$ has ~300,000 digits — we can't compute it exactly
- Pascal's triangle up to 10^6: 10^{12} cells — too big
- Goal: **O(N) precomputation, O(1) per query**

Why a modulus?

- True answers are astronomically large (counting problems)
- Mod keeps numbers in 64-bit range and the answer checkable
- $10^9 + 7$ and 998244353 are **prime**, and $< 2^{30}$:
  - sum of two values $< 2^{31}$ fits in `int` ... barely — use `long long`
  - product of two values $< 2^{60}$ fits in `long long` ✓
  - prime ⇒ every non-zero value has an **inverse**

The rules

$(a + b) \bmod m = ((a \bmod m) + (b \bmod m)) \bmod m$

$(a - b) \bmod m = ((a \bmod m) - (b \bmod m) + \mathbf{m}) \bmod m$

$(a \cdot b) \bmod m = ((a \bmod m) \cdot (b \bmod m)) \bmod m$

```
const long long MOD = 1'000'000'007;
x = (x + y) % MOD;
x = ((x - y) % MOD + MOD) % MOD;   // C++ % can be negative!
x = x * y % MOD;                   // x, y < MOD → product < 2^60
```

Division does not work this way. (6 / 2) mod 4 = 3, but (6 mod 4)/(2 mod 4) = 1.

Reduce early, reduce often

```
long long ans = 1;
for (int i = 1; i <= n; i++)
    ans = ans * i % MOD;       // n! mod p
```

- Reduce **after every** multiplication
- `ans * i * j % MOD` can overflow: reduce between factors
- C++ `%` keeps the sign of the dividend: `-7 % 5 == -2`
- Modulus up to 10^{18}? Products overflow `long long` → `__int128`

Binary exponentiation

a^b with b = 13 = 1101_2: $a^{13} = a^8 \cdot a^4 \cdot a^1$

```
long long power(long long a, long long b, long long m) {
    long long r = 1 % m;
    a %= m;
    while (b > 0) {
        if (b & 1) r = r * a % m;   // use this bit
        a = a * a % m;              // a, a², a⁴, a⁸, ...
        b >>= 1;
    }
    return r;
}
```

O(log b) multiplications. b = 10^{18} → 60 steps. (CSES 1095)

Trace: $3^{13} \bmod 1000$

b (binary)	bit	r	a
1101	1	3	9
110	0	3	81
11	1	243	6561 % 1000 = 561
1	1	243·561 % 1000 = 323	…

$3^{13} = 1{,}594{,}323 \rightarrow$ **323** ✓

Modular inverse

x is an inverse of a mod m if **$a \cdot x \equiv 1 \pmod{m}$**.
Then "divide by a" = "multiply by x".

- Exists $\Leftrightarrow$ **gcd(a, m) = 1**
- m prime, a $\neq$ 0: always exists

Example mod 7: $3 \cdot 5 = 15 \equiv 1 \rightarrow 3^{-1} = 5$.
So $4 / 3 \equiv 4 \cdot 5 = 20 \equiv 6 \pmod{7}$. Check: $3 \cdot 6 = 18 \equiv 4$ ✓

Fermat's little theorem

p prime, a not divisible by p:

$a^{(p-1)} \equiv 1 \pmod p$ ⇒ $a \cdot a^{(p-2)} \equiv 1$ ⇒ **$a^{-1} = a^{(p-2)}$**

```
long long inv(long long a) { return power(a, MOD - 2, MOD); }
```

- O(log p) ≈ 30 multiplications
- Requires p **prime**, a ≠ 0
- Non-prime m with gcd(a, m) = 1 → extended Euclid (Regionals track)
- inv(2) mod 10^9+7 = 500000004

Inverses of 1..n in O(n)

```
inv[1] = 1;
for (int i = 2; i <= n; i++)
    inv[i] = (MOD - (MOD / i) * inv[MOD % i] % MOD) % MOD;
```

Why: write $p = q \cdot i + r$, so $q \cdot i + r \equiv 0 \Rightarrow i \equiv -r/q$

$\Rightarrow i^{-1} \equiv -q \cdot r^{-1}$ with $r = p \bmod i < i$ (already known).

Handy when you need many inverses, e.g. harmonic sums mod p.

Binomial coefficients

C(n, k) = n! / (k! (n−k)!) = number of k-subsets of an n-set

Precompute once, O(N):

```
fact[0] = 1;
for (int i = 1; i <= N; i++) fact[i] = fact[i-1] * i % MOD;
invFact[N] = power(fact[N], MOD - 2, MOD);
for (int i = N; i > 0; i--) invFact[i-1] = invFact[i] * i % MOD;

long long nCr(int n, int k) {
    if (k < 0 || k > n) return 0;
    return fact[n] * invFact[k] % MOD * invFact[n-k] % MOD;
}
```

One exponentiation, then O(1) per query.

Why `invFact[i-1] = invFact[i] * i`?

$1/(i-1)! = i / i!$

So from the inverse of N! we walk down to 0! with one multiplication each.
Avoids N separate `power` calls (N log p → N).

Requires **N < p**: for n ≥ p, n! ≡ 0 mod p and the formula breaks
→ Lucas' theorem (out of scope today).

Alternative: Pascal's triangle

$C(n, k) = C(n-1, k-1) + C(n-1, k)$

```
for (int n = 0; n <= N; n++) {
    C[n][0] = 1;
    for (int k = 1; k <= n; k++)
        C[n][k] = (C[n-1][k-1] + C[n-1][k]) % m;
}
```

- $O(N^2)$ time and memory → N ≤ ~5000
- Works for **any** modulus m (no inverses needed!)
- Also fine with no modulus while values < 2^{63} (N ≤ 66 safe)

Counting toolbox

Count	Formula	CSES
k-subsets of n	$C(n, k)$	1079
lattice paths in an $a \times b$ grid	$C(a + b, a)$	—
n identical balls into k boxes	$C(n + k - 1, k - 1)$	1716
arrangements of a multiset	$n! / (c_1! \, c_2! \, \dots)$	1715
balanced bracket sequences, $2n$ chars	Catalan $C(2n, n)/(n + 1)$	2064

Division by a factorial = multiplication by `invFact`.

Worked example: Distributing Apples (CSES 1716)

n children, m identical apples. Ways to distribute?

Stars and bars: line up m stars and n − 1 bars;
bars split the stars into n (possibly empty) groups.

```
** | | *          n = 3, m = 3  →  (2, 0, 1)
```

Answer = C(m + n − 1, n − 1). For n = 3, m = 2: C(4, 2) = 6 ✓

Exponent towers: CSES 1712

Compute a^(b^c) mod p, all values up to 10^9.

By Fermat, a^e only depends on e **mod (p − 1)** when p ∤ a:

```
long long e = power(b, c, MOD - 1);   // NOT mod MOD!
long long ans = power(a, e, MOD);
```

Edge: if p | a, the answer is 0 unless the true exponent is 0.
3^(7^1) = 2187, 15^(2^2) = 50625 ✓

Modular arithmetic struct (optional)

```
struct Mint {
    int v;
    Mint(long long x = 0) { x %= MOD; if (x < 0) x += MOD; v = x; }
    Mint& operator+=(Mint o) { v += o.v; if (v >= MOD) v -= MOD; return *this; }
    Mint& operator*=(Mint o) { v = (long long)v * o.v % MOD; return *this; }
    // -, /, pow, inv ... (see notes)
};
```

- Removes "forgot to `% MOD`" bugs
- Write it once, test it, keep it in your notebook

Complexity summary

Operation	Time
+, −, × mod m	$O(1)$
a^b mod m	$O(\log b)$
inverse via Fermat	$O(\log p)$
inverses of 1..N	$O(N)$
fact / invFact tables	$O(N + \log p)$
nCr query	$O(1)$
Pascal table	$O(N^2)$

Pitfalls

- Negative results from `%` after subtraction → add MOD
- `int` overflow in `a * b` → cast to `long long` **before** multiplying
- Forgetting to reduce the base: `power(a, ...)` with a ≥ MOD or a < 0
- Using Fermat with a **non-prime** modulus, or with a ≡ 0
- Reducing an exponent mod p instead of **p − 1**
- `nCr` with k > n or k < 0 must return 0, not crash
- Tables sized for n but queried with n + m − 1

Practice problems

- CSES 1095 — Exponentiation
- CSES 1712 — Exponentiation II
- CSES 1079 — Binomial Coefficients
- CSES 1716 — Distributing Apples
- CSES 1715 — Creating Strings II
- CSES 2064 — Bracket Sequences I
- AtCoder ABC 145 D — Knight
- AtCoder ABC 156 D — Bouquet
- Codeforces 300C — Beautiful Numbers

Summary

- Reduce after every operation; fix negatives with `+ MOD`
- Binary exponentiation: O(log b), the workhorse of modular math
- Prime modulus ⇒ a^{-1} = a^(p−2); exponents reduce mod p − 1
- fact + invFact ⇒ nCr in O(1) after O(N) setup
- Stars and bars, multinomials, Catalan: all just factorial ratios
- Next week: **stacks, queues & monotonic structures**